



ĐẠI HỌC BÁCH KHOA HÀ NỘI

TRƯỜNG CÔNG NGHỆ THÔNG TIN
VÀ TRUYỀN THÔNG

NHẬP MÔN CÔNG NGHỆ SONG SINH THỰC - SỐ VÀ ỨNG DỤNG

Bài 2.1

SỐ HÓA ĐA PHƯƠNG TIỆN

Các thuật toán mã hóa nén không mất thông tin với dữ liệu văn bản

ONE LOVE. ONE FUTURE.

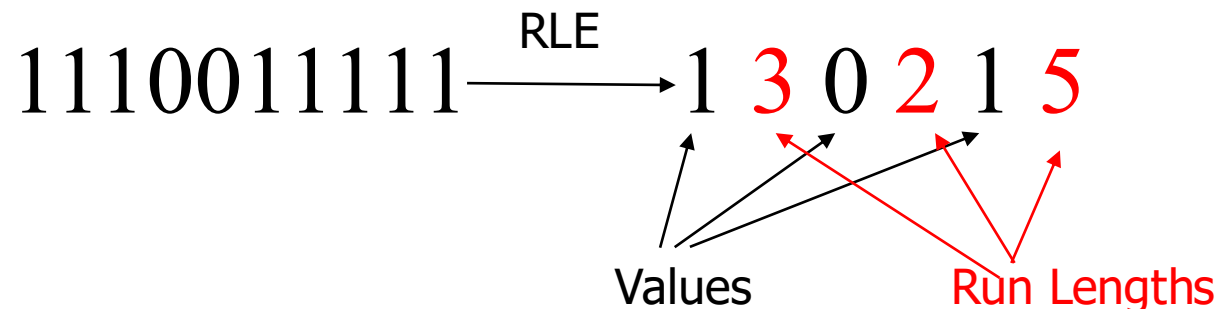
Các thuật toán mã hóa nén không mất thông tin với dữ liệu văn bản

- Runlength Encoding
- Huffman
- Lempel – Ziv Welch
- Mã hóa nén số học

Run Length Encoding (RLE)

- Kỹ thuật nén

- Biểu diễn dữ liệu dựa trên giá trị xuất hiện và số lần xuất hiện liên tiếp của giá trị trong văn bản gốc.
- Số lần xuất hiện liên tiếp của giá trị trong văn bản gốc gọi là 'run length'.



- Ứng dụng

- Hữu ích khi sử dụng để nén dữ liệu văn bản vì giá trị lặp lại xuất hiện nhiều (giá trị càng xuất hiện trùng lặp liên tiếp thì thuật toán càng hiệu quả)
- Cài đặt rất đơn giản khi so với các phương pháp nén khác
- Do là nén không mất thông tin nên dễ dàng giải nén

Ví dụ: Run Length Encoding (RLE)

- Văn bản DDDDDCCCCOODDE: chữ cái D xuất hiện 5 lần nên được mã hóa thành D5. Như vậy, thông điệp sẽ được nén thành D5C4O3D2E1 (10 kí tự thay vì 15 kí tự).
- Với văn bản chỉ chứa số như sau: 1111111111122 sẽ được mã hóa thành 11222. Chữ số 1 có thể xuất hiện 1222 lần, hoặc có thể hiểu là chữ số 2 xuất hiện 22 lần => chúng ta cần dùng dấu cách, do đó văn bản mã hóa trở thành 1 12 2 2.
- Với văn bản chỉ chứa số như sau: 123456 sẽ được mã hóa thành 1 1 2 1 3 1 4 1 5 1 6 1.
 - Văn bản ban đầu có 6 kí tự, trong khi văn bản mã hóa có 18 kí tự (tính cả dấu cách trắng)
 - trường hợp tồi: văn bản mã hóa có kích thước lớn hơn cả văn bản chưa mã hóa

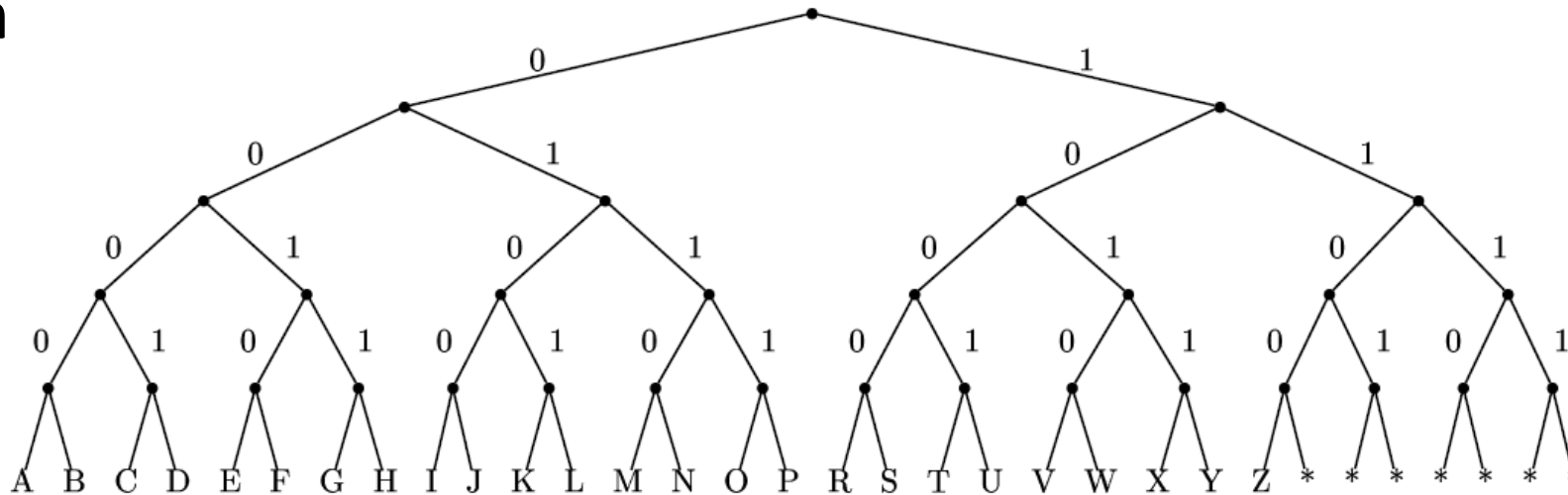
Mã hoá độ dài cố định

- Để mã hoá 26 chữ cái tiếng Anh bởi mã nhị phân độ dài cố định, độ dài của xâu tối thiểu là $\lceil \log 26 \rceil = 5$ bit.

00001	A	01000	H	01111	O	10110	V
00010	B	01001	I	10000	P	10111	W
00011	C	01010	J	10001	Q	11000	X
00100	D	01011	K	10010	R	11001	Y
00101	E	01100	L	10011	S	11010	Z
00110	F	01101	M	10100	T		
00111	G	01110	N	10101	U		

- Các xâu từ 11011 đến 11111 không sử dụng

- Cây mã hoá độ dài cố định



- Cây mã hóa độ dài cố định là cây nhị phân đầy đủ
- Giải mã 0011101000 → HI
- Các nút * không sử dụng

Mã Huffman

- Tư tưởng là tạo cây mã hóa có thể không phải cây nhị phân đầy đủ
- Giả sử có một văn bản trên bảng chữ cái C . Với mỗi chữ cái $c \in C$ ta biết tần suất xuất hiện của nó trong văn bản là $f(c)$. Cần tìm cách mã hoá văn bản sử dụng ít bộ nhớ nhất.
- Mã phi tiền tố (prefix free code) : là cách mã hóa ký tự sao cho mã của một ký tự bất kỳ không là đoạn đầu của bất kỳ mã của một ký tự nào trong số các ký tự còn lại.

Mã Huffman

- Mỗi mã phi tiền tố có thể biểu diễn bởi một cây nhị phân T mà mỗi lá của nó tương ứng với một chữ cái và cạnh của nó được gán cho một trong hai số 0 hoặc 1.
- Mã của một chữ cái c là 1 dãy nhị phân gồm các số gán cho các cạnh trên đường đi từ gốc đến lá tương ứng với c .
- Ý tưởng thuật toán:
 - Chữ cái có tần suất nhỏ hơn cần được gán cho lá có khoảng cách đến gốc là lớn hơn; chữ cái có tần suất xuất hiện lớn hơn cần được gán cho nút gần gốc hơn

Mã Huffman: Bước mã hóa

```

procedure Huffman(C, f);
begin
   $n \leftarrow |C|$ ;
   $Q \leftarrow C$ ;
  for  $i:=1$  to  $n-1$  do
    begin
       $x, y \leftarrow$  2 chữ cái có tần suất nhỏ nhất trong  $Q$ ; (* Thao tác 1 *)
      Tạo nút  $p$  với hai con  $x, y$ ;
       $f(p) := f(x) + f(y)$ ;
       $Q \leftarrow Q \setminus \{x, y\} \cup \{p\}$  (* Thao tác 2 *)
    end;
  end;

```

- Có thể cài đặt với thời gian $O(n \log n)$ sử dụng priority queue



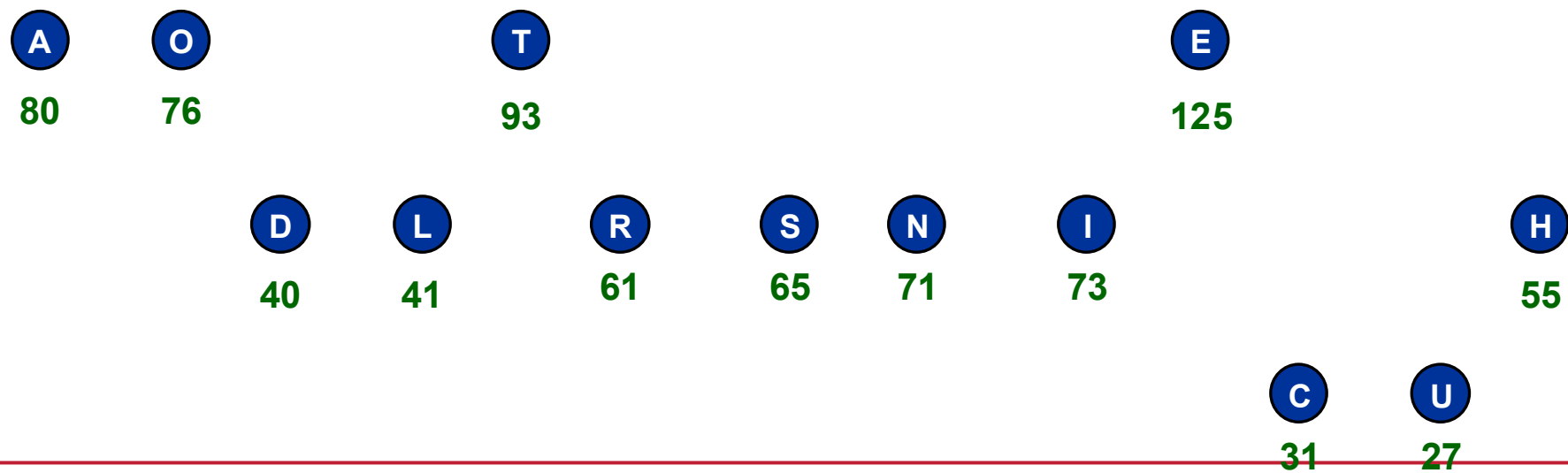
HUST

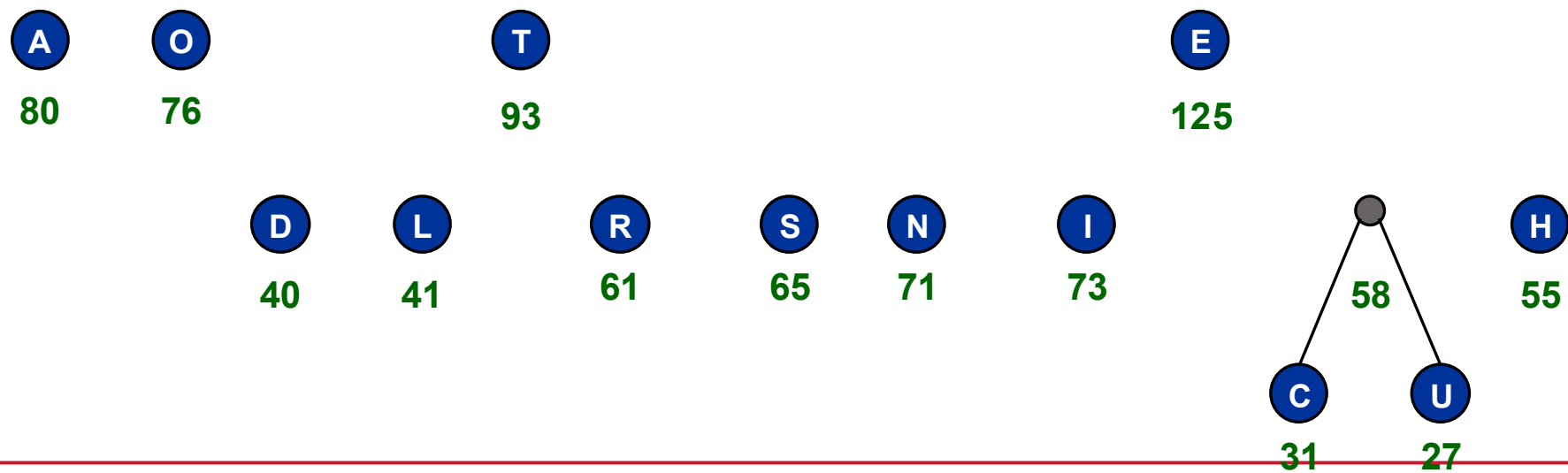
Ví dụ

- Tần suất của các ký tự trong văn bản.

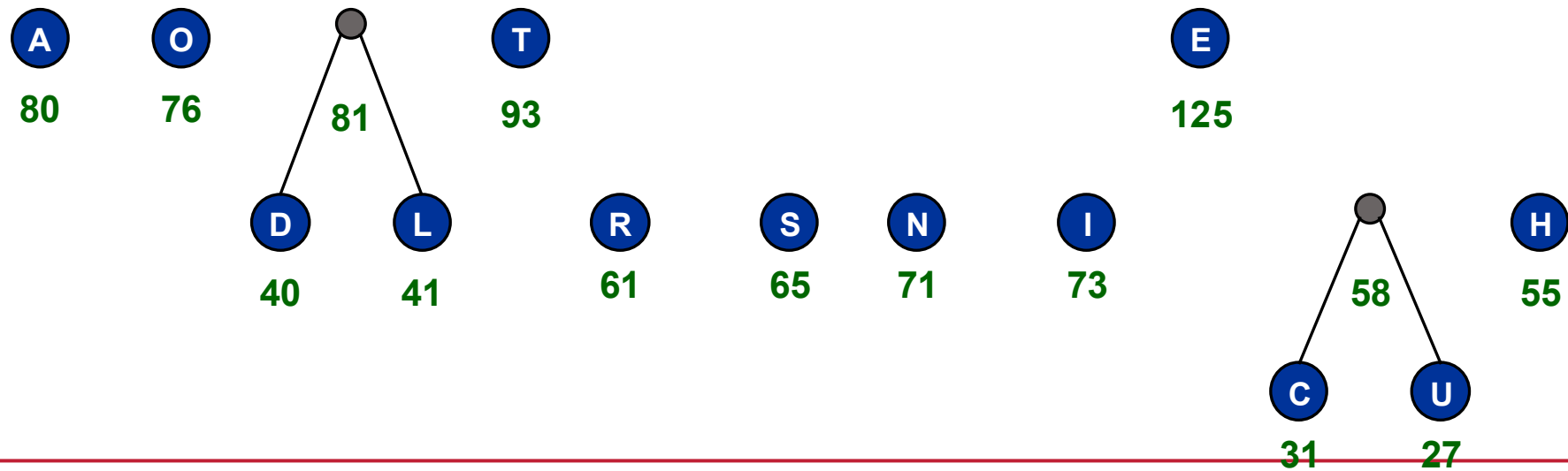
Char	Freq
E	125
T	93
A	80
O	76
I	72
N	71
S	65
R	61
H	55
L	41
D	40
C	31
U	27

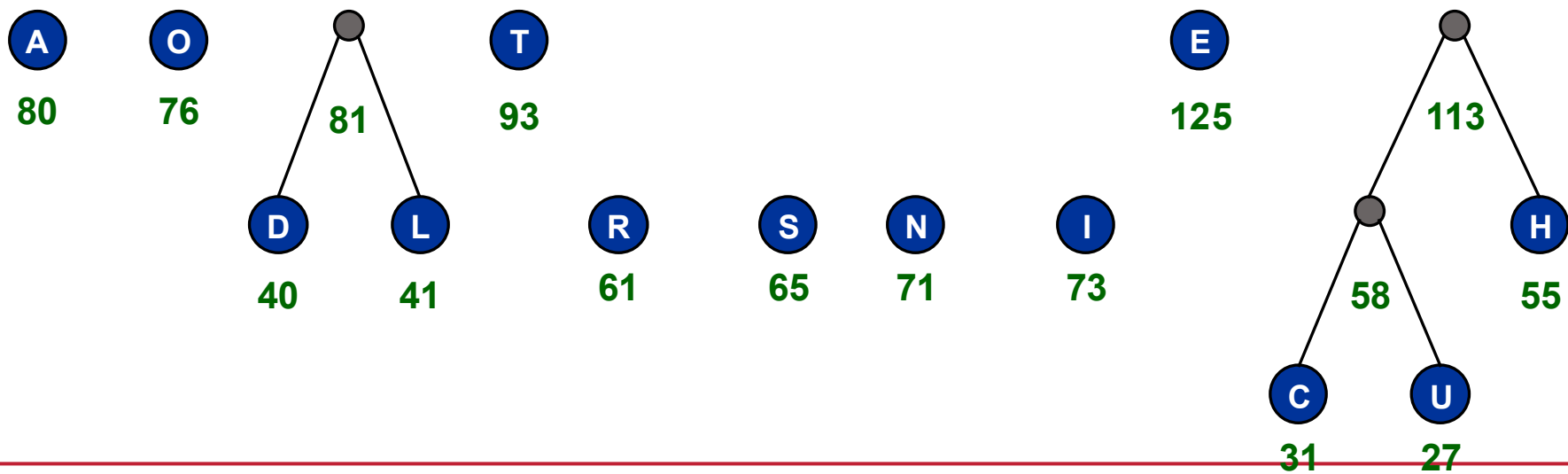


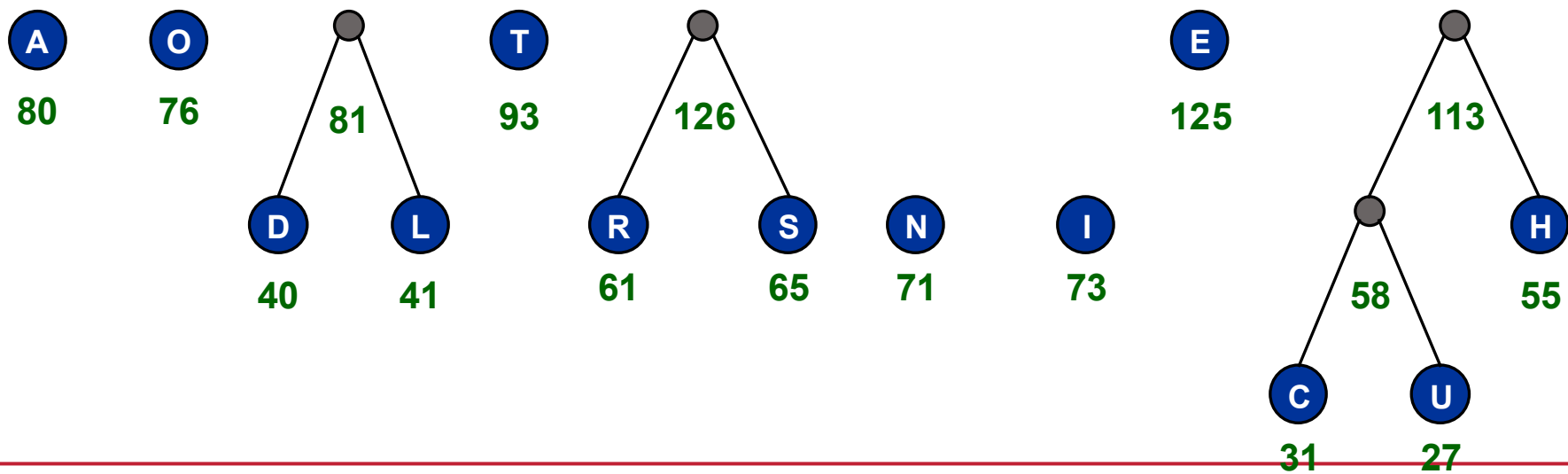


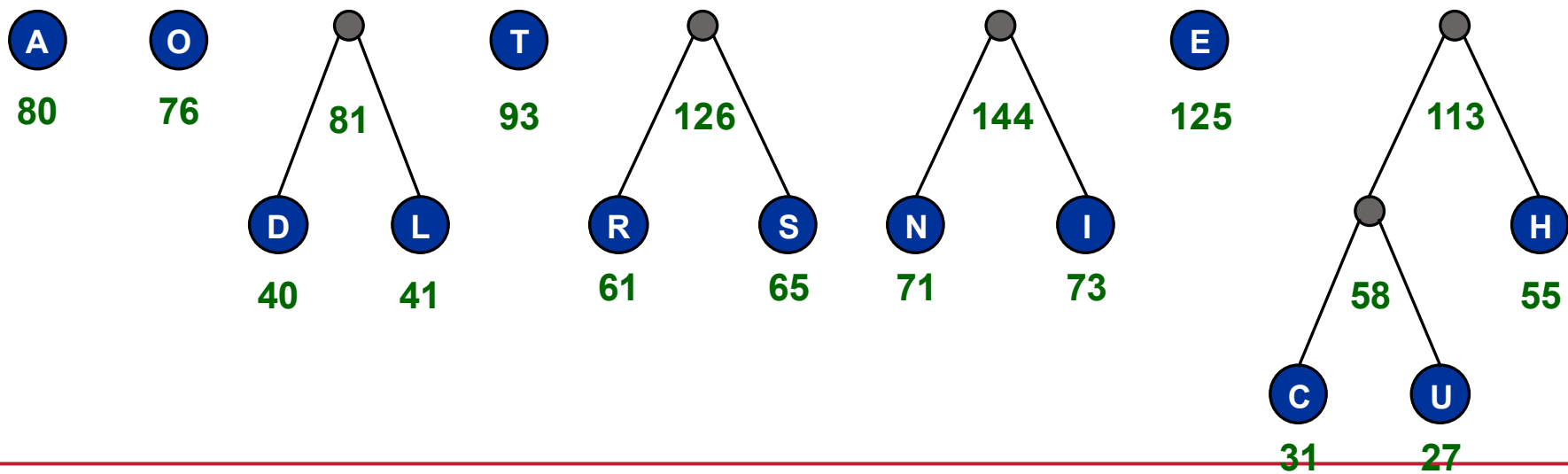


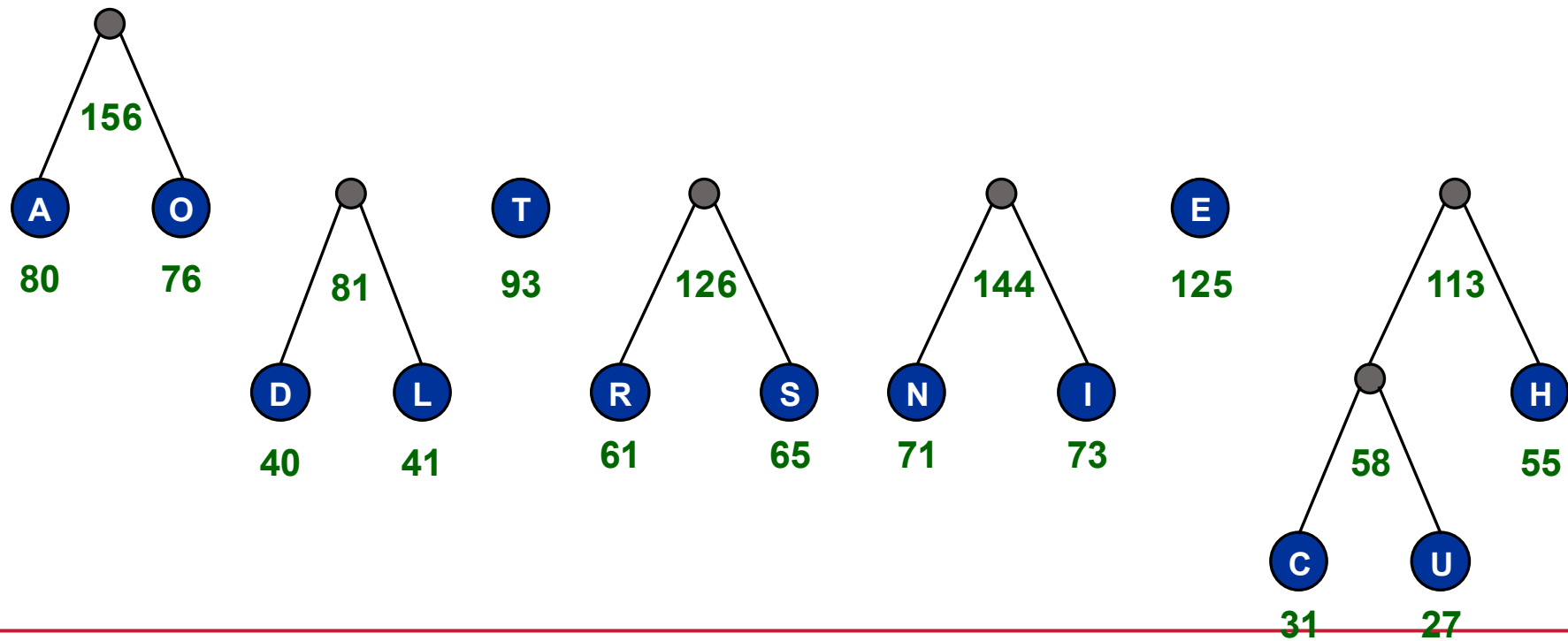
Xây dựng mã Huffman

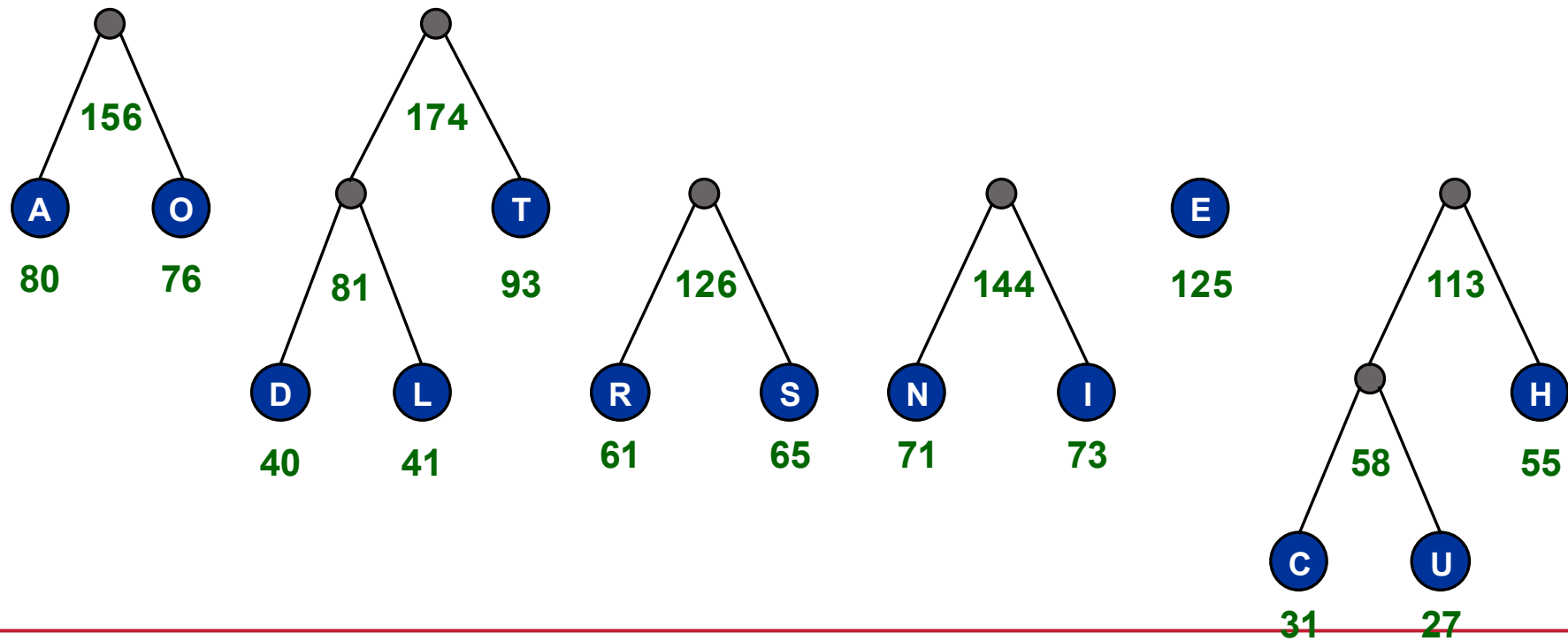


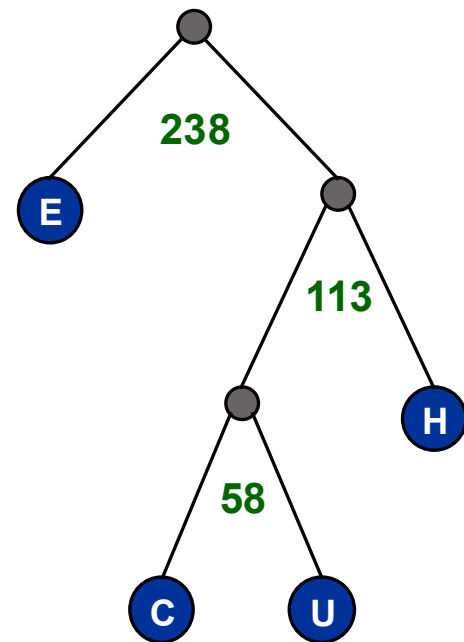
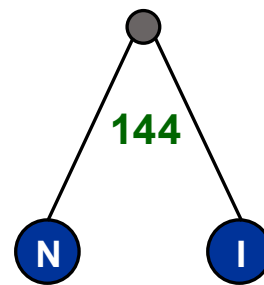
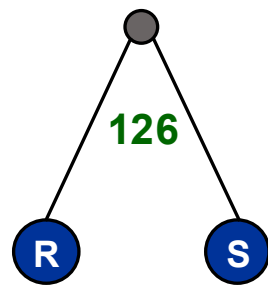
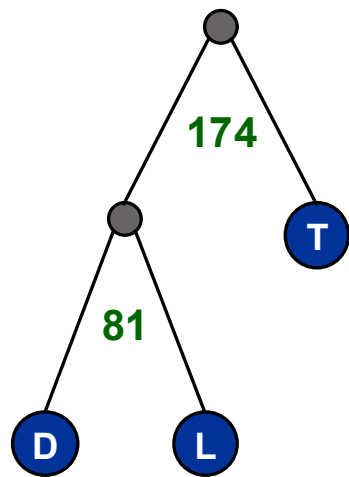
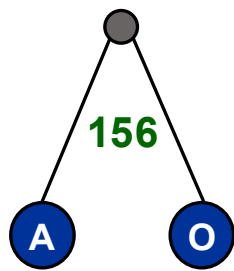


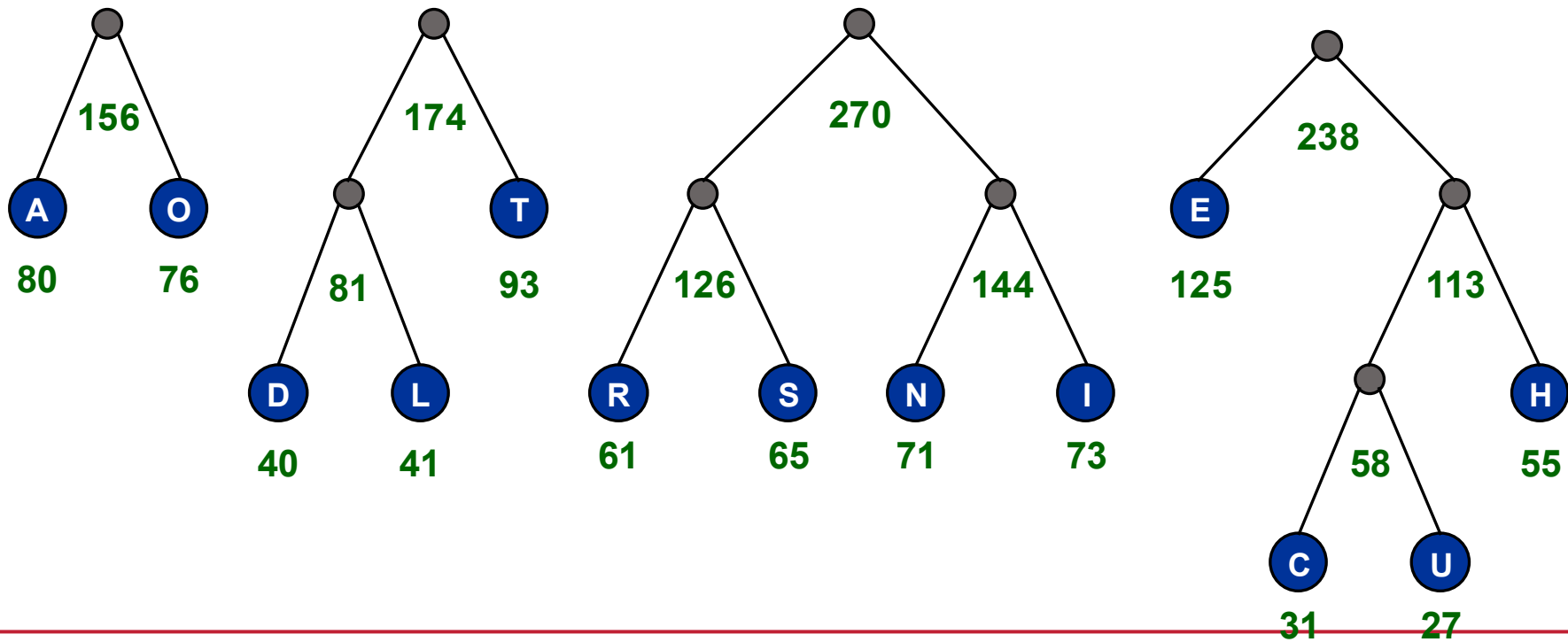


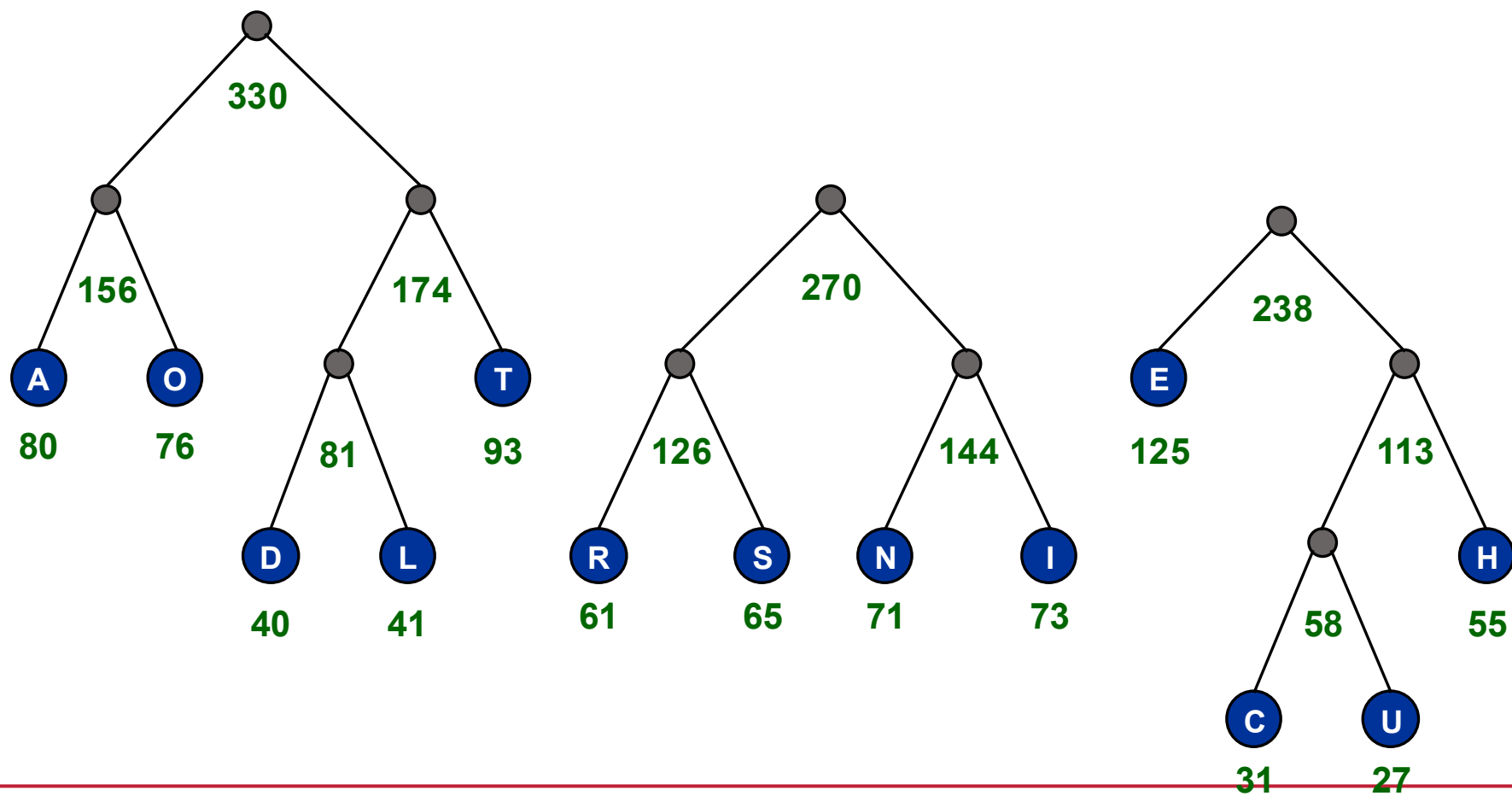


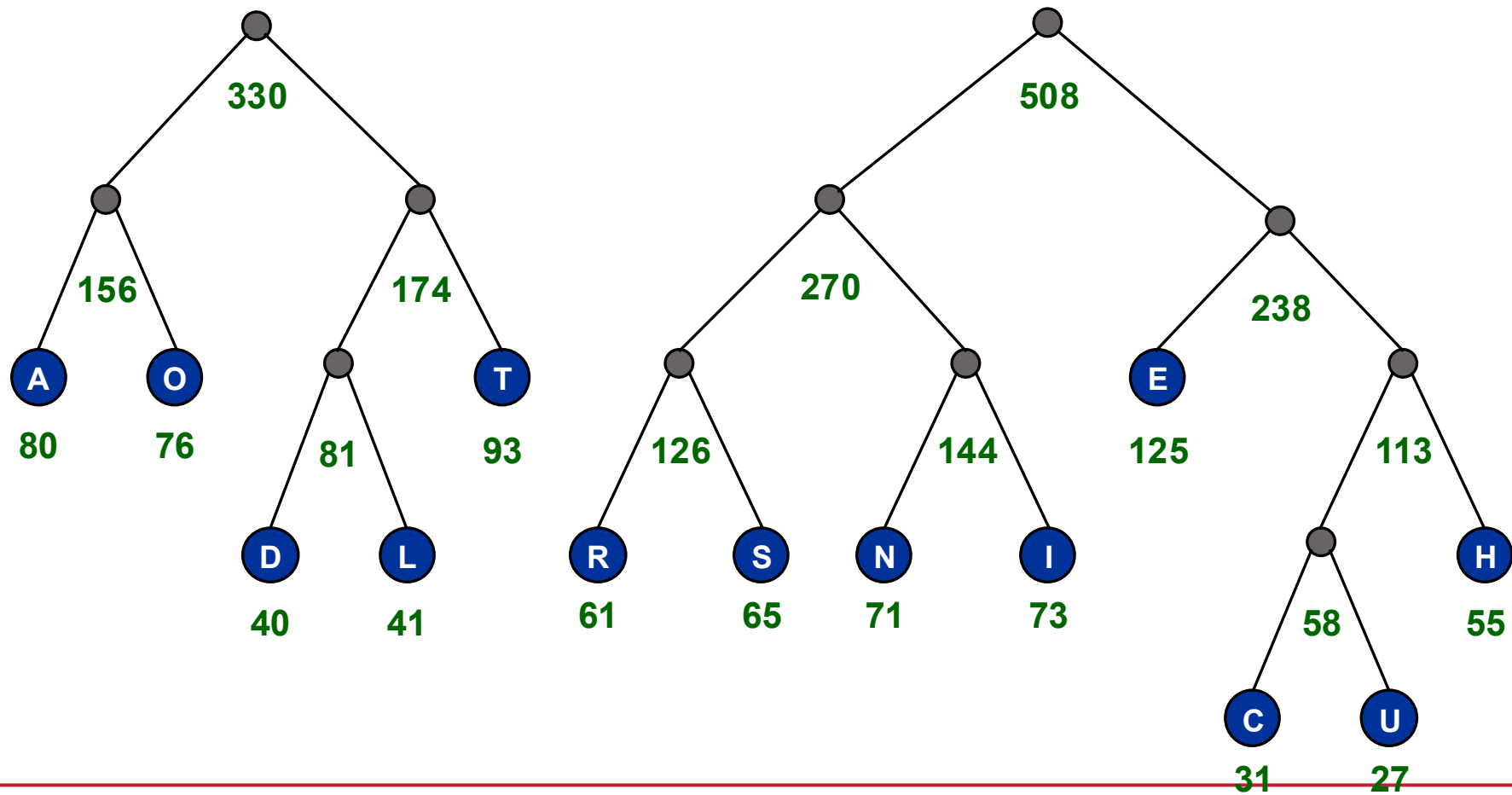


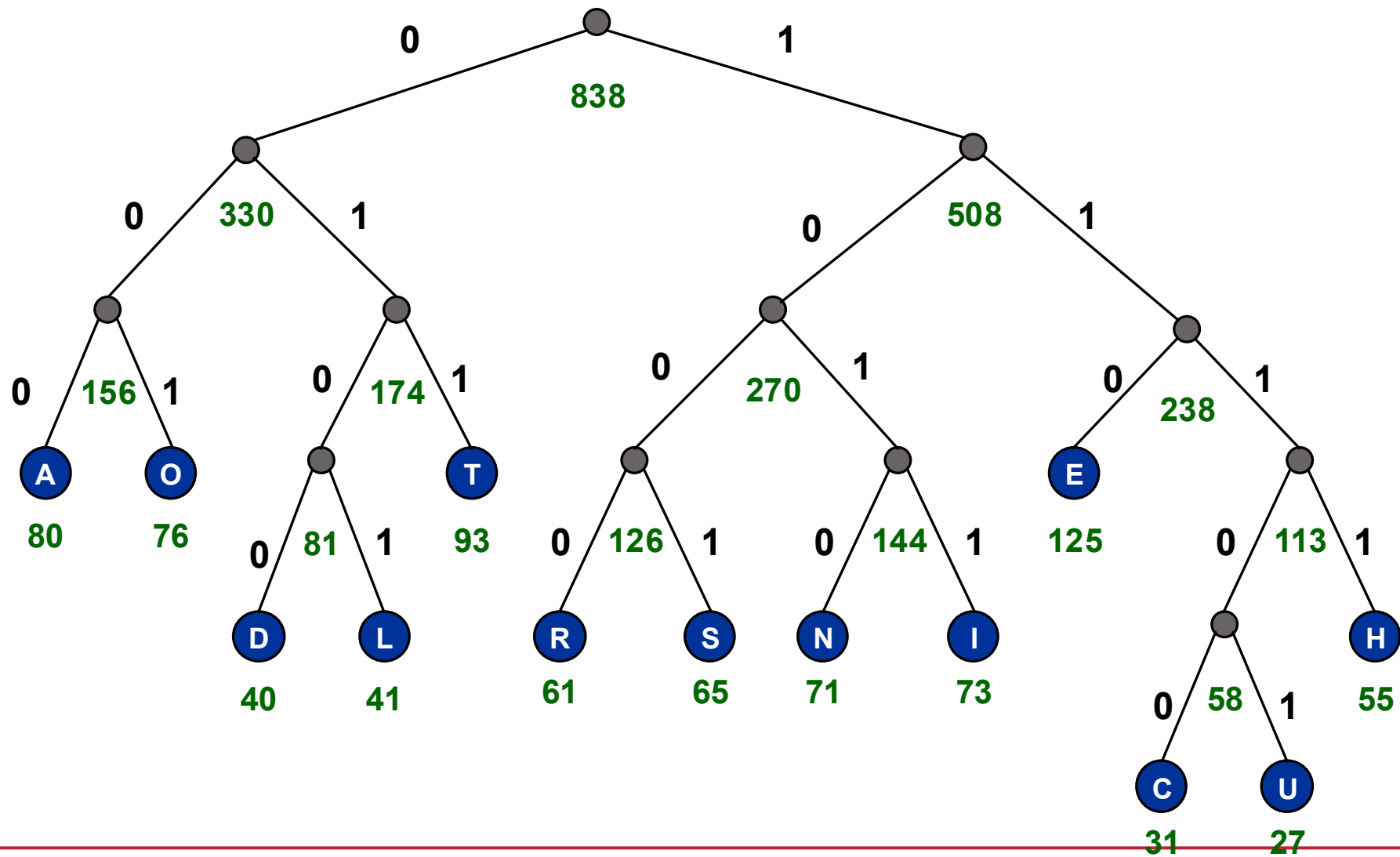












Xây dựng mã Huffman

Char	Freq	Fixed	Huff
E	125	0000	110
T	93	0001	011
A	80	0010	000
O	76	0011	001
I	73	0100	1011
N	71	0101	1010
S	65	0110	1001
R	61	0111	1000
H	55	1000	1111
L	41	1001	0101
D	40	1010	0100
C	31	1011	11100
U	27	1100	11101
Tổng	838	3352	3036

Mã hóa Huffman: Giải mã

```

procedure Huffman_Decode(B);
(* B là xâu mã hóa văn bản theo mã hóa Huffman. *)
begin
  <Khai báo P là gốc của cây mã hóa Huffman>
  While <chưa là bit kết thúc của B> do
    begin
      x ← bit tiếp theo trong xâu B;
      If x = 0 then      P ← Con trái P
      Else              P ← Con phải của P
      If (P là nút lá) then
        begin
          <Hiển thị ký hiệu tương ứng với nút lá>
          <Đặt lại P là gốc của cây mã hóa Huffman>
        end;
      end
    end;
  end;

```

- Ưu điểm:
 - Dựa trên tần suất xuất hiện của các ký tự
 - Giải quyết được sự dư thừa phân bố ký tự
- Nhược điểm:
 - Đòi hỏi phải xây dựng cây nhị phân
 - Quá trình giải nén phức tạp do không biết trước chiều dài mã cho đến khi ký tự đầu tiên được in ra
 - Phải gửi cả bảng mã cho phía nhận mới giải nén được => hiệu suất nén không cao, chỉ cao khi độ lớn tập tin lớn

- Phương pháp nén được phát minh bởi Lempel - Zip và Welch.
 - Hoạt động dựa trên một ý tưởng rất đơn giản là người mã hoá và người giải mã cùng xây dựng bảng mã.
 - Bảng mã không cần được lưu kèm với dữ liệu trong quá trình nén, mà khi giải nén, người giải nén sẽ xây dựng lại nó.
- Nguyên tắc hoạt động:
 - Một chuỗi kí tự là một tập hợp từ hai kí tự trở lên.
 - Nhớ tất cả các chuỗi kí tự đã gặp và gán cho nó một dấu hiệu (gọi là token) riêng.
 - Nếu lần sau gặp lại chuỗi kí tự đó, chuỗi kí tự sẽ được thay thế bằng token đã gán cho nó.

- Phương pháp nén cần phải tạo một “từ điển” rất lớn dùng để lưu giữ các chuỗi ký tự đã gặp
- Khi duyệt các dữ liệu văn bản, các ký tự được giữ lại trong một bộ đệm tạm thời (gọi là Accumulator) và đem so sánh với các chuỗi ký tự đã có trong "từ điển".
- Nếu chuỗi dữ liệu trong bộ đệm chứa không có trong "từ điển" thì nó được bổ sung thêm vào "từ điển" và được cấp một token.
- Nếu chuỗi trong bộ đệm chứa đã có trong "từ điển" thì token đã gán cho chuỗi sẽ được hiển thị ra ngoài.

- Có bốn qui tắc để thực hiện việc nén dữ liệu theo thuật toán LZW là:
 1. qui tắc 1: 256 token đầu tiên từ 0 (0) đến 0ffh (255) được dành cho các kí tự đơn.
 2. qui tắc 2: So sánh với "từ điển" khi trong bộ đệm tạm thời đã có nhiều hơn một kí tự.
 3. qui tắc 3: Các kí tự ở đầu vào được bổ sung vào bộ đệm chứa cho đến khi chuỗi kí tự trong bộ đệm tạm thời không có trong "từ điển".
 4. qui tắc 4: Khi bộ đệm tạm thời có một xâu kí tự mà trong "từ điển" không có thì xâu trong bộ đệm tạm thời được cho vào "từ điển". Kí tự cuối cùng của xâu kí tự trong bộ đệm tạm thời được giữ lại trong bộ đệm để tiếp tục tạo thành xâu kí tự mới.



Ví dụ

- Ví dụ: Các bước để mã hoá chuỗi "!BAN!BA!BAA!BAR!BAT" như sau:
 - Bước 1: Kí tự thứ nhất '!' được cất vào bộ đệm tạm thời để chuẩn bị tạo nên một xâu kí tự.
 - Bước 2: Kí tự thứ hai 'B' nối thêm vào sau kí tự "!". Vì trong "từ điển" chưa có chuỗi "!B" nên chuỗi này được thêm vào "từ điển" và được gán dấu hiệu là 100h (Vì từ 000h đến 0ffh được dành riêng cho các kí tự đơn: Qui tắc 1). '!' được gửi ra còn 'B' phải ở lại trong bộ đệm tạm thời.



Ví dụ

- Bước 3: Kí tự thứ ba 'A' thêm vào sau 'B'. Chuỗi "BA" cũng chưa có trong "từ điển" nên nó được thêm vào "từ điển" và gán token là 101h. 'A' ở lại trong bộ đệm tạm thời còn 'B' được đưa ra.
- Bước 4: Kí tự thứ tư 'N' thêm vào sau 'A' tạo thành chuỗi "AN" cũng chưa có trong "từ điển" nên được thêm vào "từ điển" và gán token là 102h. 'N' ở lại trong bộ đệm tạm thời còn 'A' được đưa ra.
- Bước 5: Kí tự thứ năm '!' thêm vào sau 'N' để tạo thành chuỗi "N!", "N!" được thêm vào "từ điển" với token là 103h. '!' ở lại còn 'N' được đưa ra.

Ví dụ

- Bước 6: Kí tự thứ sáu 'B' thêm vào sau '!'. Lần này thì chuỗi "!B" đã có trong "từ điển" nên không có kí tự nào được đưa ra. "!B" tiếp tục ở lại trong bộ đệm để tạo ra xâu mới.
- Bước 7: Kí tự 'A' thêm vào sau 'B' để tạo thành xâu "!BA", do "!BA" không có trong "từ điển" nên nó được thêm vào "từ điển" và gán token là 104h đồng thời token 100h được gửi ra thay cho "!B" (Quy tắc 4). "A" tiếp tục ở lại trong bộ đệm để chờ tạo thành xâu mới.
- Các bước trên cứ thế tiếp tục cho đến khi hết tập tin cần nén. Việc giảm kích thước chỉ thực sự bắt đầu tại bước 7 khi mà một token (độ lớn khoảng 12 bits) là <100h> được đưa ra thay cho xâu kí tự "!B" (độ lớn khoảng 2 byte).

Ví dụ

STT	Bộ đệm chứa	Dữ liệu vào (8 Bits)	Dữ liệu ra (12 Bits)	Từ điển
1	-	!	-	-
2	!	B	!	100h=!B
3	B	A	B	101h=BA
4	A	N	A	102h=AN
5	N	!	N	103h=N!
6	!	B	-	-
7	!B	A	<100h>	104h=!BA
8	A	!	A	105h=A!
9	!	B	-	-

Ví dụ

10	!B	A	-	-
11	!BA	A	<104h>	106h=!BAA
12	A	!	-	-
13	A!	B	<105h>	107h=A!B
14	B	A	-	-
15	BA	R	<101h>	108h=BAR
16	R	!	R	109h=R!
17	!	B	-	-
18	!B	A	-	-
19	!BA	T	<104h>	10Ah=!BAT
20	-	-	T	-

- Phần lớn thời gian khi bắt đầu nén chủ yếu mất vào việc tạo "từ điển".
- Khi "từ điển" đủ lớn, xác suất gặp nhau ở bộ đệm trong "từ điển" tăng lên và càng nén được nhiều hơn.
- Với mỗi một token, ta phải lưu một chuỗi trong "từ điển" để so sánh. Vì token được biểu diễn bằng một số 12 bits nên "từ điển" sẽ có 4096 token, khi tăng số bit để biểu diễn token lên thì hiệu quả nén sẽ tốt hơn nhưng lại tăng dung lượng lưu trữ.

Mã hóa LZW

```
read a character k
w = k;
while (read a character k )
{
    if wk exists in the dictionary
        w = wk;
    else
        add wk to the dictionary;
        if w contains a character
            output character
        else
            output the code for w;
        w = k;
}
```

Giải nén LZW

```
read a character k;
  output k;
  w = k;
  while ( read k )
  /* k could be a character or a code. */
  {
    if k is a character or code k exists in the dictionary
      entry = character k or string in dictionary for code k;
      output entry;
      add w + firstCharacterOf(entry) to dictionary;
      w = entry;
    else
      output entry = w + firstCharacterOf(w);
      add entry to dictionary;
      w = entry;
  }
```


Ví dụ khác của Nén LZW

w	k	Dictionary	Output
-	b	-	-
b	o	bo (256)	98 (b)
o	o	oo (257)	111 (o)
o	o	-	-
oo	p	oop (258)	257 (oo)
p	-	-	112 (p)

- Ưu điểm:
 - Tỷ lệ nén tương đối cao, trong tập tin nén không cần phải chứa bảng mã
 - Bên nhận có thể tự xây dựng bảng mã mà không cần bên gửi phải gửi kèm theo bản tin nén
 - Thuật toán LZW khắc phục được sự lãng phí về bộ nhớ mà các thuật toán trước không tận dụng được hết. Đồng thời khắc phục được sự cứng nhắc của thuật toán nén, góp phần làm thuật toán nén linh hoạt hơn.
- Nhược điểm: – Tốn nhiều bộ nhớ, khó thực hiện trên các mảng bé hơn 64KB

Vấn đề của mã hóa nén Huffman

Huffman Coding

- ☐ Replacing an input symbol with a codeword
- ☐ Need a probability distribution
- ☐ Hard to adapt to changing statistics
- ☐ Need to store the codeword table
- ☐ Minimum codeword length is 1 bit



Mã hóa Số học - Arithmetic Coding

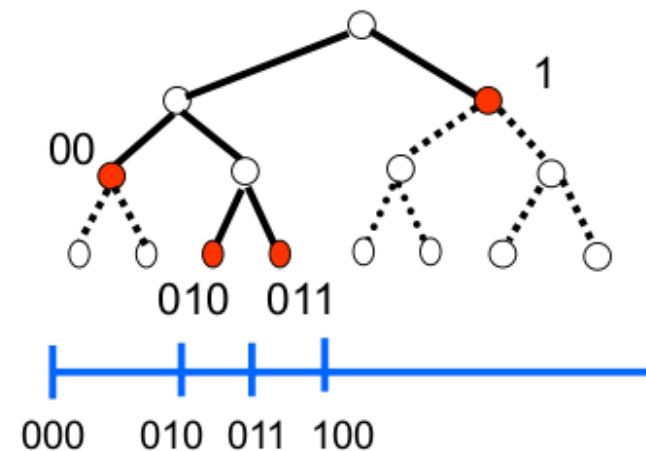
- Replace the entire input with a single floating-point number
- Does not need the probability distribution
- Adaptive coding is very easy
- No need to keep and send codeword table



Mã hóa Số học - Arithmetic Coding

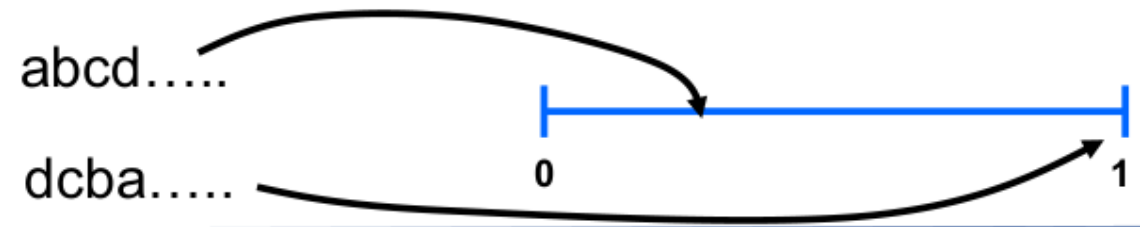
□ Recall table look-up decoding of Huffman code

- N: alphabet size
- L: Max code word length
- Divide $[0, 2^L]$ into N intervals
- One interval for one symbol
- Interval size is **roughly** proportional to symbol prob.



Mã hóa Số học - Arithmetic Coding

- Arithmetic coding applies this idea **recursively**
 - Normalizes the range $[0, 2^L]$ to $[0, 1]$.
 - Map an input **sequence** (multiple symbols) to a unique **tag** in $[0, 1)$.



Mã hóa Số học - Arithmetic Coding

❑ Symbol set and prob: a (0.8), b(0.02), c(0.18)

❑ Disjoint and complete partition of the range $[0, 1)$
 $[0, 0.8)$, $[0.8, 0.82)$, $[0.82, 1)$

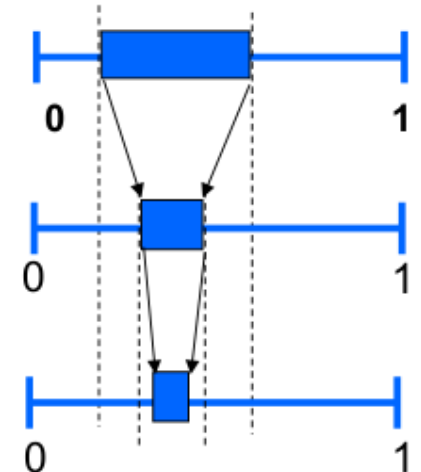
❑ Each interval corresponds to one symbol

❑ Interval size is proportional to symbol probability



■ The first symbol restricts the tag position to be in one of the intervals

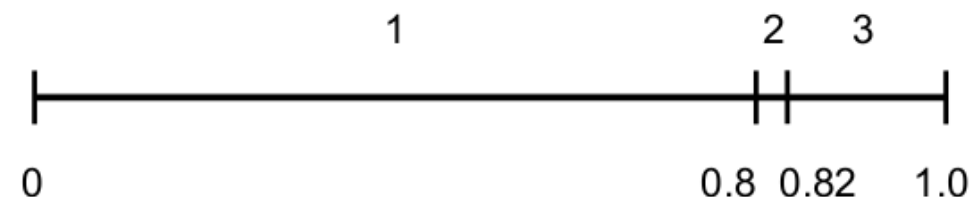
■ The reduced interval is partitioned recursively as more symbols are processed.



■ Observation: once the tag falls into an interval, it never gets out of it

Mã hóa Số học - Arithmetic Coding

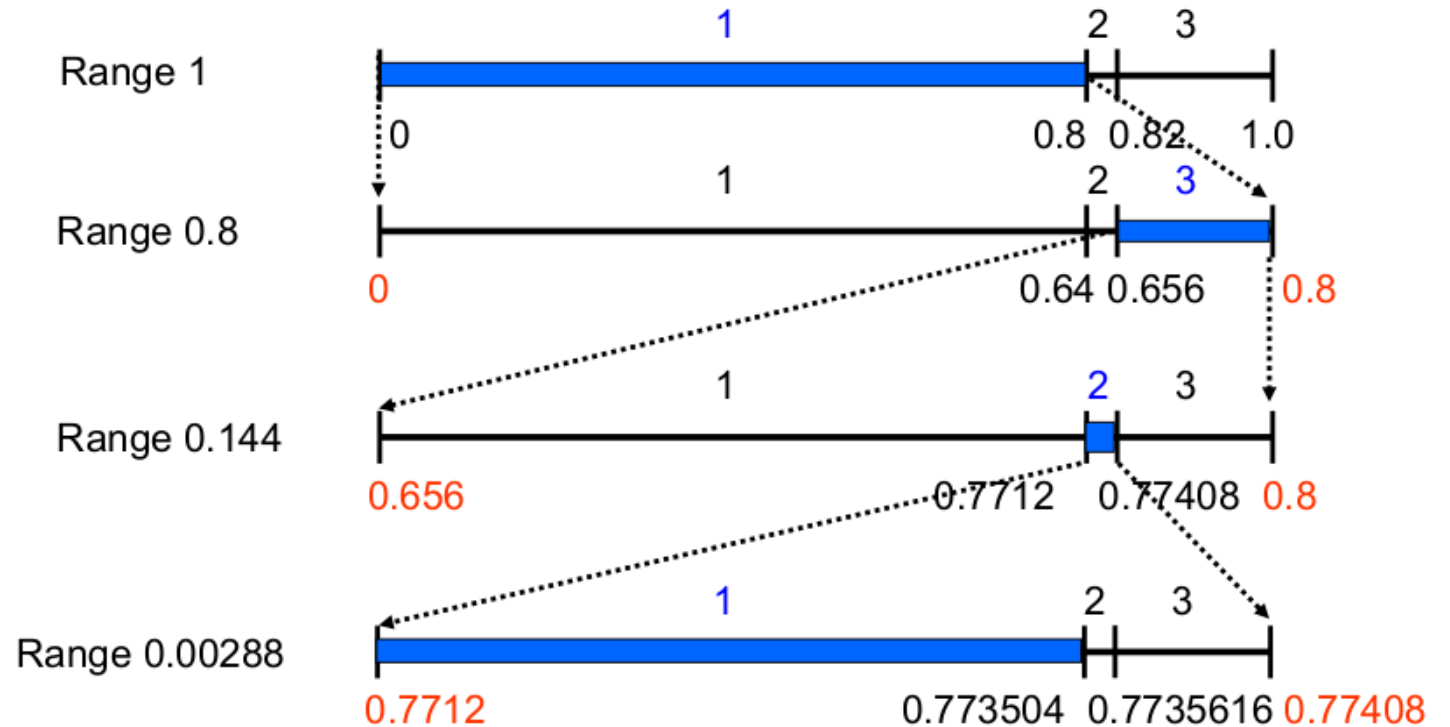
Symbol	Prob.
1	0.8
2	0.02
3	0.18



- Map to real line range $[0, 1)$
- Order does not matter
 - Decoder needs to use the same order
- Disjoint but complete partition:
 - 1: $[0, 0.8)$: 0, 0.799999...9
 - 2: $[0.8, 0.82)$: 0.8, 0.819999...9
 - 3: $[0.82, 1)$: 0.82, 0.999999...9

Mã hóa Số học - Arithmetic Coding

■ Input sequence: "1321"



Termination: Encode the lower end or midpoint to signal the end.

- Difficulties:**
1. Shrinking of interval requires very **high precision** for long sequence.
 2. No output is generated until the entire sequence has been processed.

Mã hóa Số học - Arithmetic Coding

- Keep track of
- **LOW, HIGH, RANGE**
- Any two are sufficient, e.g., LOW and RANGE.

```

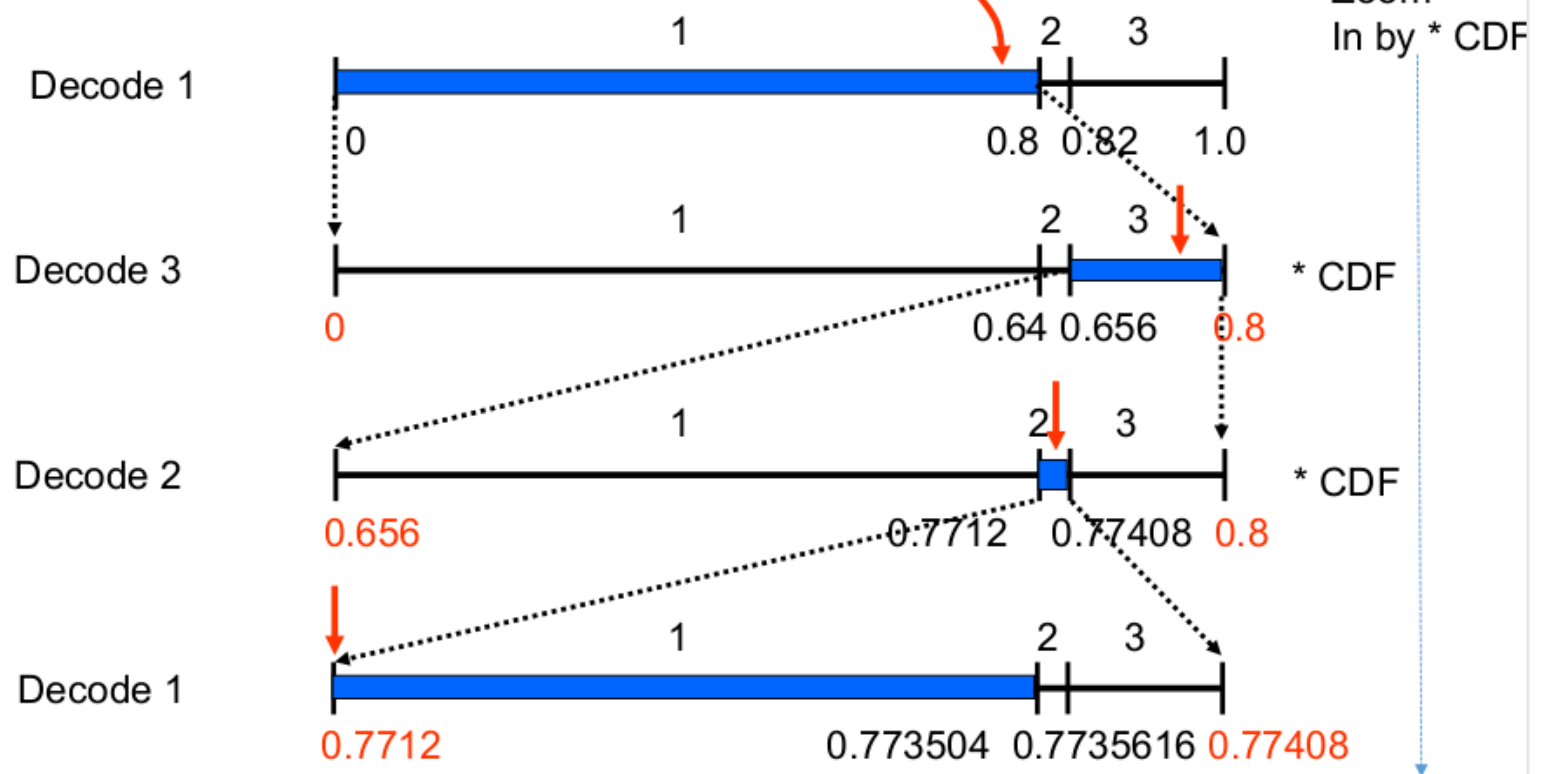
LOW=0.0, HIGH=1.0;
while (not EOF) {
    n = ReadSymbol();
    RANGE = HIGH - LOW;
    HIGH = LOW + RANGE * CDF(n);
    LOW = LOW + RANGE * CDF(n-1);
}
output LOW;
    
```

Input	LOW	HIGH	RANGE
Initial	0.0	1.0	1.0
1	$0.0 + 1.0 * 0 = 0.0$	$0.0 + 1.0 * 0.8 = 0.8$	0.8
3	$0.0 + 0.8 * 0.82 = 0.656$	$0.0 + 0.8 * 1 = 0.8$	0.144
2	$0.656 + 0.144 * 0.8 = 0.7712$	$0.656 + 0.144 * 0.82 = 0.77408$	0.00288
1	$0.7712 + 0.00288 * 0 = 0.7712$	$0.7712 + 0.00288 * 0.8 = 0.773504$	0.002304

Mã hóa Số học - DeCoding

□ Arithmetic Decoding (conceptual)

Suppose encoder encodes the lower end: 0.7712



Drawback: need to recalculate all thresholds each time.

Mã hóa Số học - DeCoding

- Normalize RANGE to [0, 1) each time
- No need to recalculate the thresholds.

$$x \leftarrow \frac{x - low}{range}$$

Receive 0.7712

Decode 1

$$x = (0.7712 - 0) / 0.8$$

$$= 0.964$$

Decode 3

$$x = (0.964 - 0.82) / 0.18$$

$$= 0.8$$

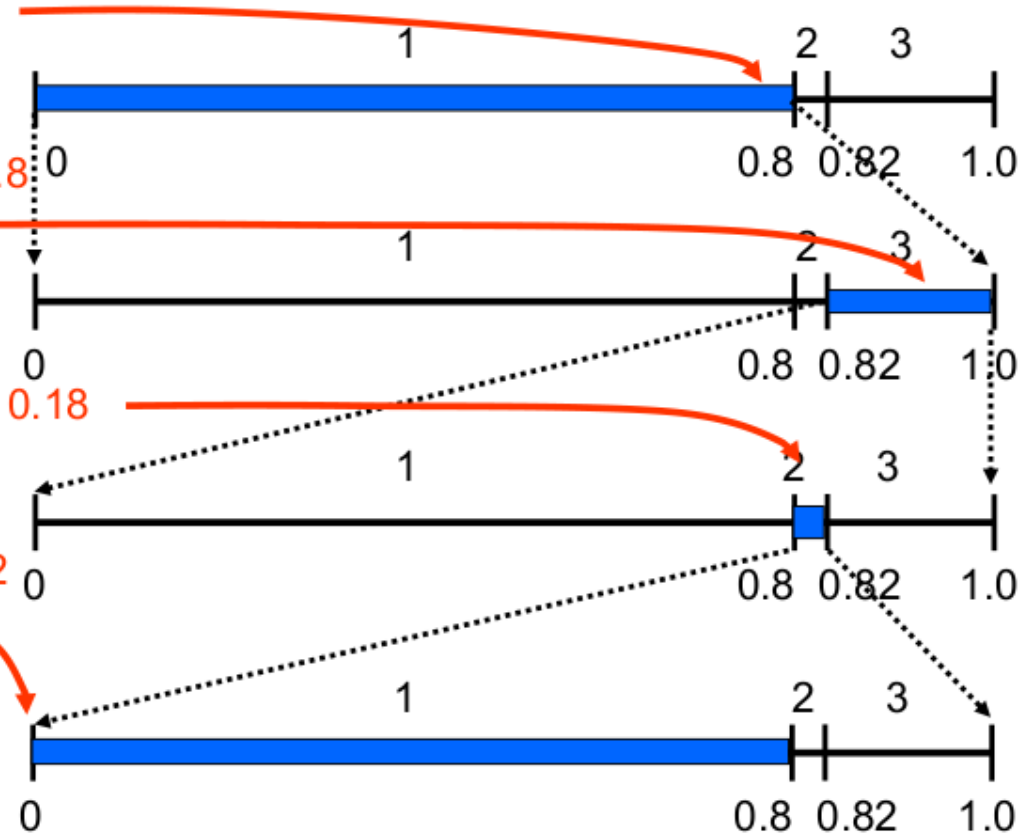
Decode 2

$$x = (0.8 - 0.8) / 0.02$$

$$= 0$$

Decode 1.

Stop.

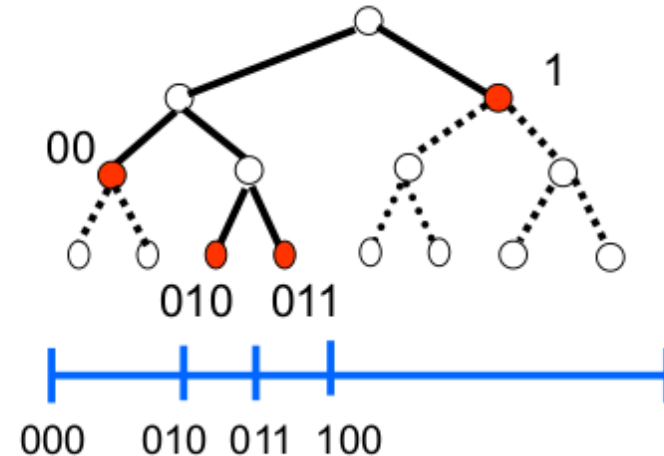


Mã hóa Số học - DeCoding

```

Low = 0; high = 1;
x = GetEncodedNumber();
While (x ≠ low) {
    n = DecodeOneSymbol(x);
    output symbol n;
    x = (x - CDF(n-1)) / (CDF(n) - CDF(n-1));
};

```



But this method still needs high-precision floating point operations.
It also needs to read all input at the beginning of the decoding.

Mã hóa Số học – Ưu và nhược điểm

- Ưu điểm:
 - Tỷ lệ nén tốt hơn so với nhiều thuật toán mã hóa nén entropy khác
- Nhược điểm:
 - Khó cài đặt

- Các thuật toán hỗ trợ có thể dùng kèm với các thuật toán vừa nêu để tăng tỷ lệ nén:
 - Move to Front Transform
 - Burrows-Wheeler Transform
- Ví dụ thuật toán nén Bzip2 là một thuật toán nén dữ liệu được sử dụng để giảm kích thước của các file. Nó hoạt động bằng cách sử dụng hai bước chính: biến đổi Burrows-Wheeler Transform và mã hóa Huffman

Move to Front Transform

- Move to Front (MTF) transform is an encoding of data (typically a stream of bytes) designed to improve the performance of entropy encoding (coding scheme that assigns codes to symbols so as to match code lengths with the probabilities of the symbols) techniques of compression.
- When properly implemented, it is fast enough that its benefits usually justify including it as an extra step in data compression algorithms.

Move to Front Transform

- In the context of MTF each byte value is encoded by its index in a list, which changes over the course of the algorithm.
- The list is initially stored, e.g., in order by byte value (0, 1, 2, 3, ..., 255). Therefore, the first byte is always encoded by its own value.
- However, after encoding a byte, that value is moved to the front of the list before continuing to the next byte.

Move to Front Transform - Example

Iteration	Sequence	List
b ananaaaa	1	(abcdefghijklmnopqrstuvwxyz)
ba n anaaaa	1,1	(bacdefghijklmnopqrstuvwxyz)
ban a naaaa	1,1,13	(abcdefghijklmnopqrstuvwxyz)
banan a aaa	1,1,13,1	(nabcdefghijklmnopqrstuvwxyz)
bananab a aa	1,1,13,1,1	(anbcdefghijklmnopqrstuvwxyz)
bananabab a a	1,1,13,1,1,1	(nabcdefghijklmnopqrstuvwxyz)
bananababab a	1,1,13,1,1,1,0	(anbcdefghijklmnopqrstuvwxyz)
bananabababab a	1,1,13,1,1,1,0,0	(anbcdefghijklmnopqrstuvwxyz)
Final	1,1,13,1,1,1,0,0	(anbcdefghijklmnopqrstuvwxyz)



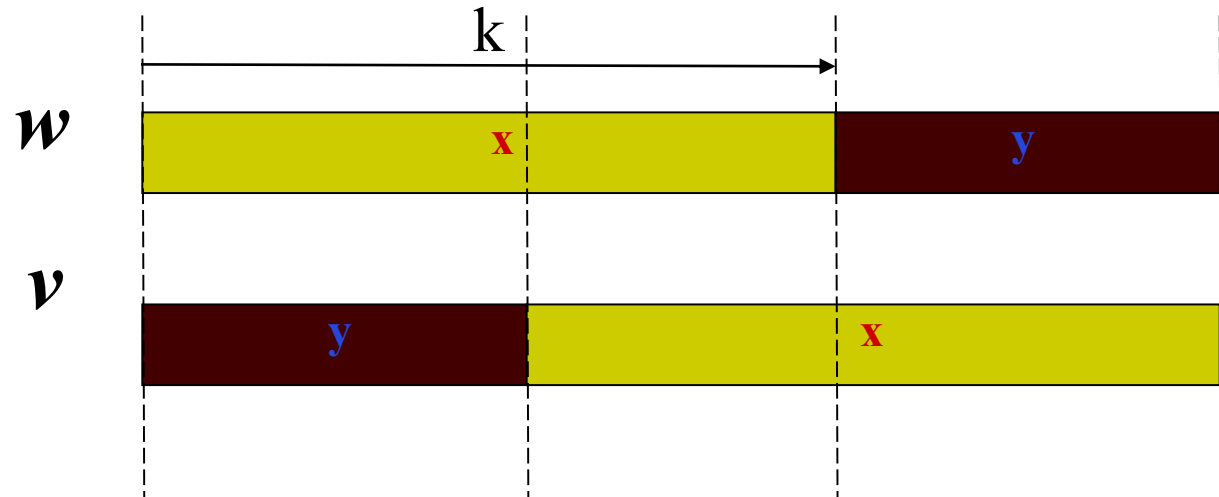
Burrows-Wheeler Transform

- The Burrows-Wheeler transform (BWT), a.k.a. block-sorting compression, is one of the most popular method in data compression.
- It was invented by Michael Burrows and David Wheeler, in 90-ties.
- When a character string is transformed by the BWT, none of its characters change value. The transform rearranges in clever for the order of the characters in the string.
- If the original string had several substrings that occurred frequently, then the transformed string will have several places where a single character is repeated multiple times in a row.
- This is useful for compression, since it tends to be easy to compress a string that has runs of repeated characters by techniques such as move-to-front transform and run-length encoding.



Cyclic rotations

For $0 \leq k \leq n-1$, the k^{th} cyclic rotation of the string $w = w[0..n-1]$ is another string $v = v[0..n-1]$, s.t., $v[i] = w[(i+k) \bmod n]$

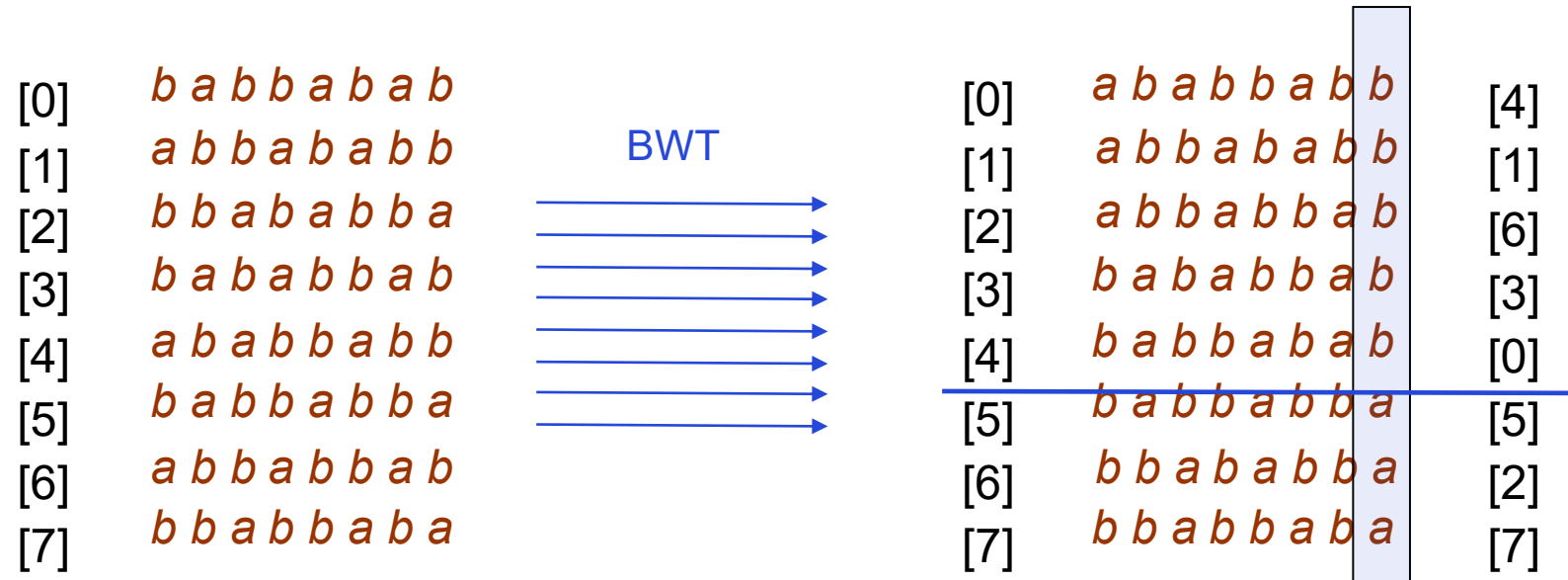


Burrows-Wheeler Transform

- The *Burrows-Wheeler Transform* transform for the string $w = w[0..n-1]$ is defined as follows:
 - Create a square matrix $M[n \times n]$ in which the k^{th} row contains the k^{th} cyclic rotation of w
 - Sort rows of M in lexicographic order
 - Store the string represented by the last column of M
 - And the index of row which contains the position of the original string w (i.e., 0^{th} cyclic rotation)

Burrows-Wheeler Transform (example)

- Consider 5th Fibonacci word $f_5 = babbabab$



- The output string *bbbbbbaaaa* and position [4]

Burrows-Wheeler Transform

- The Burrows-Wheeler transform can be computed by the algorithm that constructs suffix arrays
- Which means that the Burrows-Wheeler transform can be computed in linear time
- The Burrows-Wheeler transform is reversible and the original string can be recovered efficiently via generation of consecutive columns of matrix M

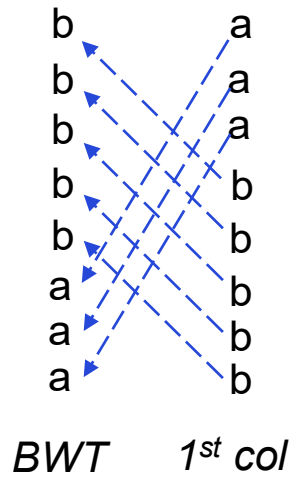
Burrows-Wheeler (reverse) Transform -- hard way

a		b	ba	ab		b	bab	aba.....	b	baba	abab.....	b	babab
a		b	ba	ab		b	bab	abb.....	b	babb	abba.....	b	babba
a		b	ba	ab		b	bab	abb.....	b	babb	abba.....	b	babba
b		b	bb	ba		b	bba	bab.....	b	bbab	baba.....	b	bbaba
b		b	bb	ba		b	bba	bab.....	b	bbab	babb.....	b	bbabb
b		a	ab	ba		a	aba	bab.....	a	abab	babb.....	a	ababb
b		a	ab	bb		a	abb	bba.....	a	abba	bbab.....	a	abbab
b		a	ab	bb		a	abb	bba.....	a	abba	bbab.....	a	abbab

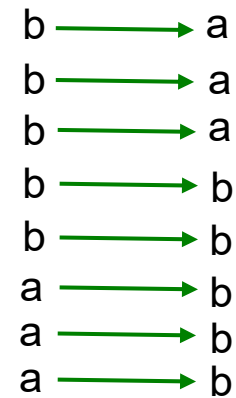
ababb	b	bababb	ababba....	b	bababba	ababbab..	b	bababbab	ababbabb
abbab	b	babbab	abbaba....	b	babbaba	abbabab..	b	babbabab	abbababb
abbab	b	babbab	abbabb....	b	babbabb	abbabba..	b	babbabba	abbabbab
babab	b	bbabab	bababb....	b	bbababb	bababba..	b	bbababba	bababbab
babba	b	bbabba	babbab....	b	bbabbab	babbaba..	b	bbabbaba	babbabab
babba	a	ababba	babbab....	a	ababbab	babbabb..	a	ababbabb	babbabba
bbaba	a	abbaba	bbabab....	a	abbabab	bbababb..	a	abbababb	bbababba
bbabb	a	abbabb	bbabba....	a	abbabba	bbabbab..	a	abbabbab	bbabbaba

Burrows-Wheeler (reverse) Transform -- easy way based on stable sorting property

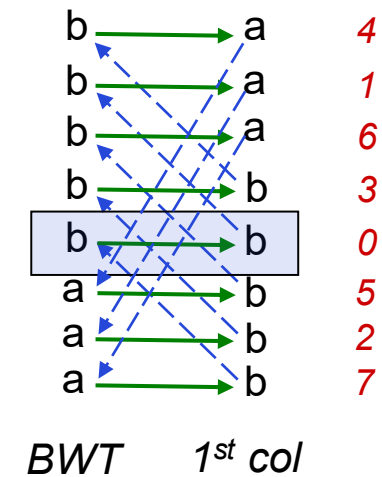
Corresponding symbols



Structure



Reverse BWT



Just follow the cycle

b a b b a b a b
0 1 2 3 4 5 6 7